

WEEK – 6

Static Fields and Methods, `this` Reference, `final` Keyword, String Handling

PROGRAM 6 – STRINGS (CO1)

1. Title

Program to Find the Longest Substring Without Repeating Characters

2. Objective / Aim

To write and execute a Java program to find the **longest substring without repeating characters** using Java String handling methods and loops.

3. Problem Statement

Write a Java program that accepts a string as input and finds the longest substring in which no character is repeated.

Example 1

Input:

abcabcbb

Output:

Length: 3
String: abc

Example 2

Input:

pwwkew

Output:

Length: 3
String: wke

Note: `pwke` is not a substring because its characters are not consecutive in the original string. It is a subsequence.

4. Concepts Used

- Classes and Objects
 - Static methods
 - String handling
 - `String.length()`
 - `String.charAt()`
 - `String.indexOf()`
 - String concatenation
 - Nested `for` loops
 - Conditional statements
 - Console input using `Scanner`
-

5. Algorithm / Logic Description

1. Start the program.
 2. Read a string from the user using `Scanner`.
 3. Create a static method named `findLongestSubstring()`.
 4. Initialize an empty string named `longest`.
 5. Use an outer loop to select each character as the starting position.
 6. For each starting position, initialize an empty string named `current`.
 7. Use an inner loop to examine the characters following the starting position.
 8. Check whether the current character already exists in `current` using `indexOf()`.
 9. If the character is already present, stop processing the current substring.
 10. Otherwise, add the character to `current`.
 11. Compare the length of `current` with `longest`.
 12. If `current` is longer, store it as `longest`.
 13. Repeat the process for all possible starting positions.
 14. Display the longest substring and its length.
 15. Stop the program.
-

6. Source Code

```
package recordprograms;

import java.util.Scanner;

public class LongestSubstring {

    // Method to find longest substring
    static void findLongestSubstring(String str) {

        String longest = "";

        for (int i = 0; i < str.length(); i++) {

            String current = "";

            for (int j = i; j < str.length(); j++) {

                char ch = str.charAt(j);

                // Check whether character is already present
                if (current.indexOf(ch) != -1) {
                    break;
                }

                current = current + ch;

                // Update longest substring
                if (current.length() > longest.length()) {
                    longest = current;
                }
            }
        }

        System.out.println("Longest substring: " + longest);
        System.out.println("Length: " + longest.length());
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        findLongestSubstring(str);

        sc.close();
    }
}
```

7. Explanation of Important String Methods

`length()`

Returns the number of characters in the string.

`str.length()`

`charAt()`

Returns the character at a particular index.

`str.charAt(j)`

`indexOf()`

Checks whether a character is already present in the current substring.

`current.indexOf(ch)`

If the character is not present, it returns -1.

String Concatenation

The statement:

`current = current + ch;`

adds the character to the current substring.

8. Sample Output

Example 1

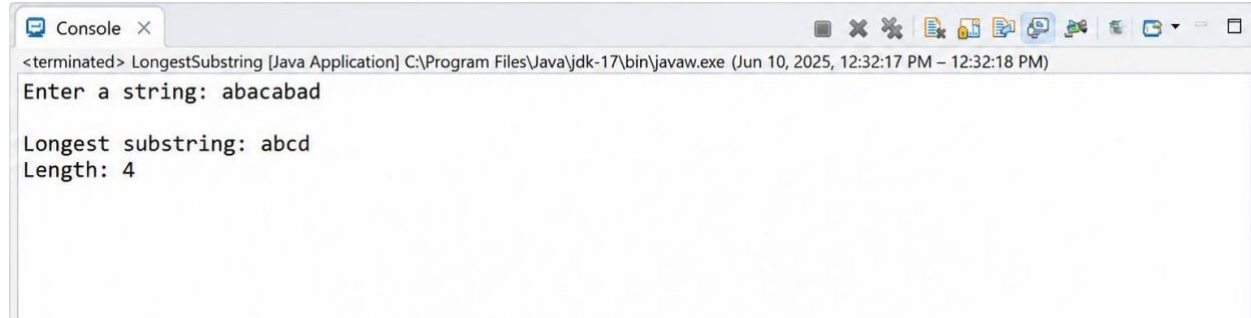
```
Enter a string: abcabcb  
Longest substring: abc  
Length: 3
```

Example 2

```
Enter a string: pwwkew  
Longest substring: wke  
Length: 3
```

Example 3

```
Enter a string: abcdef  
Longest substring: abcdef  
Length: 6
```



```
<terminated> LongestSubstring [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (Jun 10, 2025, 12:32:17 PM - 12:32:18 PM)
Enter a string: abacabad
Longest substring: abcd
Length: 4
```

9. Compilation and Execution

Save the program as:

```
LongestSubstring.java
```

Since the program uses the package `recordprograms`, compile it from the parent directory:

```
javac recordprograms/LongestSubstring.java
```

Run it using:

```
java recordprograms.LongestSubstring
```

10. Result

The Java program was successfully implemented to find the **longest substring without repeating characters**. The program uses String handling methods such as `length()`, `charAt()`, and `indexOf()`, along with loops and conditional statements to determine and display the longest non-repeating substring.